



RAG y khoa

Nền tảng, chunking và lưu trữ
dữ liệu đa nguồn

Báo cáo nghiên cứu
Tháng 10/2026
[Month YYYY]

Hướng dẫn đọc

Báo cáo đi từ cách hiểu RAG đến các quyết định triển khai. Thuật ngữ được giải thích khi xuất hiện; phần code nằm cuối để bạn có thể bỏ qua ở lượt đọc đầu.

Lộ trình đọc nhanh

Mới tìm hiểu: đọc mục 01–05, sau đó 07–10. Chuẩn bị thiết kế hệ thống: đọc thêm mục 13–20. Chuẩn bị thử nghiệm và pilot: đọc mục 21–24. Phụ lục dành cho lúc bắt đầu triển khai.

Mục lục theo chủ đề

- [01 · Tóm tắt điều hành](#)
- [02–04 · Nền tảng và luồng hoạt động của RAG](#)
- [05–06 · Đặc thù y khoa và dữ liệu đa nguồn](#)
- [07–11 · Chunking, ví dụ và bằng chứng nghiên cứu](#)
- [12–13 · Embedding và thiết kế retrieval](#)
- [14–16 · Kiến trúc lưu trữ, schema và chọn DB](#)
- [17–18 · Index, dung lượng và chi phí](#)
- [19–20 · Vòng đời dữ liệu và phân quyền](#)
- [21–23 · Đánh giá, bộ test và phân tích lỗi](#)
- [24 · Khuyến nghị và lộ trình](#)
- [25–29 · Phụ lục: dữ liệu mẫu, SQL, thuật ngữ](#)
- [Nguồn · 26 tài liệu nghiên cứu và tài liệu chính thức](#)

Ngày nghiên cứu: 01/10/2026. Các nhãn [S01]–[S26] là liên kết có thể bấm để mở nguồn gốc. Những kích thước chunk, ngân sách context và sơ đồ DB có ghi “đề xuất” cần được kiểm chứng trên dữ liệu thật.

01 · Tóm tắt điều hành

Đề xuất khởi đầu: xây một hệ thống tra cứu tri thức y khoa có nguồn dẫn, dùng chunk theo cấu trúc tài liệu kết hợp parent-child, tìm kiếm từ khóa và ngữ nghĩa, rồi xếp hạng lại kết quả. Lưu file gốc trong object storage; lưu tài liệu, phiên bản, chunk và quyền truy cập trong PostgreSQL; thử pgvector trước khi quyết định tách vector DB.

Đây là kiến trúc đề xuất để làm baseline — phiên bản chuẩn ban đầu dùng cho so sánh. Chưa có dữ liệu mẫu, lưu lượng, ngân sách hoặc yêu cầu triển khai của dự án, nên các con số trong báo cáo là điểm bắt đầu để đo, không phải cấu hình đã được xác nhận.

Năm kết luận quan trọng

Vấn đề	Kết luận	Hệ quả khi thiết kế
RAG là gì?	Tìm bằng chứng trước, dùng mô hình ngôn ngữ để diễn đạt sau.	Chất lượng phụ thuộc cả dữ liệu, tìm kiếm và câu trả lời.
Chunk thế nào?	Giữ trọn một ý cùng điều kiện, ngoại lệ và nguồn.	Không cắt đồng loạt mọi file theo cùng số ký tự.
DB nào phù hợp?	DB phải phục vụ cả tìm kiếm lẫn truy vết và cập nhật.	Vector store chỉ là một phần của hệ thống lưu trữ.
Điều gì đặc thù y khoa?	Đúng chủ đề chưa đủ; phải đúng đối tượng, thời điểm, đơn vị và phạm vi áp dụng.	Metadata và kiểm tra nguồn là thành phần cốt lõi.
Đánh giá ra sao?	Đo truy xuất, tính đúng của câu trả lời và khả năng từ chối riêng biệt.	Chưa có bộ câu hỏi chuẩn thì chưa thể kết luận cách chunk nào tốt nhất.

Phạm vi của báo cáo

Giả định chính là trợ lý tra cứu tài liệu cho nhóm dự án hoặc người có chuyên môn, với nguồn tiếng Việt và tiếng Anh. Các mục về hồ sơ bệnh án chỉ áp dụng nếu dự án thực sự chứa dữ liệu bệnh nhân. Hệ thống hỏi đáp tài liệu, truy vấn hồ sơ một bệnh nhân và phân tích toàn bộ tập bệnh nhân cần các luồng truy xuất khác nhau.

Điều nên chốt đầu tiên là “ai hỏi, hỏi việc gì, dựa trên nguồn nào”. Chọn vector DB hoặc một mô hình lớn hơn trước khi chốt ba câu này thường chưa giải quyết được điểm khó nhất.

02 · RAG từ nền tảng

Hình dung bằng một thư viện

Một LLM giống người có khả năng đọc và diễn đạt tốt, nhưng không bảo đảm nhớ đúng bộ tài liệu của bạn. RAG bổ sung bước tra cứu: tìm những đoạn liên quan, đưa chúng vào ngữ cảnh của câu hỏi, rồi yêu cầu mô hình trả lời dựa trên bằng chứng đó. Bài nghiên cứu nền tảng của Lewis và cộng sự kết hợp bộ truy xuất với mô hình sinh; nhiều hệ thống thực tế ngày nay triển khai các thành phần này theo cách mô-đun. [S01]

Ví dụ: người dùng hỏi “Tài liệu nội bộ nói gì về theo dõi sau thủ thuật X?”. Hệ thống tìm phần hướng dẫn tương ứng, kiểm tra phiên bản còn hiệu lực, rồi trả lời kèm tên tài liệu và trang. Không tìm được bằng chứng phù hợp thì câu trả lời đúng có thể là “chưa đủ thông tin trong kho”.

Những từ sẽ gặp xuyên suốt

Thuật ngữ	Cách hiểu dễ nhớ	Vai trò
Corpus	Toàn bộ kho tài liệu được phép dùng.	Xác định phạm vi tri thức.
Chunk	Một đơn vị nội dung nhỏ để tìm kiếm.	Cầu nối giữa tài liệu dài và câu hỏi ngắn.
Token	Đơn vị mà mô hình dùng để xử lý văn bản.	Đo giới hạn đầu vào; không bằng số từ hoặc ký tự.
Embedding	Dãy số biểu diễn nội dung cho tìm kiếm.	Giúp tìm các cách diễn đạt gần nghĩa.
Retriever	Bộ tìm kiếm ứng viên.	Tìm rộng để hạn chế bỏ sót.
Reranker	Bộ đọc lại và xếp hạng các ứng viên.	Chọn những đoạn sát câu hỏi hơn.
Context	Bằng chứng được đưa cho LLM ở lượt hỏi.	Làm cơ sở cho câu trả lời.

RAG, fine-tuning và context dài khác nhau thế nào?

RAG thay đổi thông tin được cung cấp khi hỏi. Fine-tuning thay đổi trọng số mô hình, thường phù hợp khi cần điều chỉnh hành vi hoặc năng lực chuyên biệt. Hai cách có thể kết hợp; fine-tuning không thay thế việc lưu và dẫn nguồn theo phiên bản.

Đưa toàn bộ tài liệu vào context có thể hợp lý với một tập nhỏ, nhưng phải đo chi phí, độ trễ và khả năng tìm đúng bằng chứng. Nghiên cứu Lost in the Middle cho thấy vị trí thông tin ảnh hưởng đến các mô hình được thử; context dài không tự bảo đảm sử dụng đầy đủ nội dung.

[S06]

03 · Bên trong bước tìm kiếm

Ba tín hiệu bổ sung cho nhau

Từ khóa (lexical): hữu ích khi câu hỏi có tên thuốc, mã xét nghiệm, tên viết tắt hoặc một cụm từ cần khớp chính xác. BM25 là một thuật toán xếp hạng phổ biến của nhóm này. Nếu tài liệu viết tiếng Anh nhưng câu hỏi hoàn toàn tiếng Việt, khớp từ đơn thuần có thể không đủ.

Ngữ nghĩa (dense): biến câu hỏi và chunk thành các vector, sau đó tìm vector gần nhau. “Theo dõi sau phẫu thuật” có thể gần với “postoperative monitoring” nếu mô hình hỗ trợ tốt hai ngôn ngữ. Gần nhau về vector không có nghĩa là phát biểu đúng, còn hiệu lực hoặc áp dụng cho cùng nhóm người bệnh.

Metadata: giới hạn những nguồn được xét, như tenant, quyền truy cập, loại tài liệu, ngôn ngữ và trạng thái phiên bản. Bộ lọc quyền truy cập phải do hệ thống tin cậy quyết định; LLM không được tự mở rộng phạm vi này.

Các công thức đủ dùng

Độ tương đồng cosine của hai vector q và x là: $\cos(q, x) = (q \cdot x) / (\|q\| \times \|x\|)$. Đây là độ gần trong không gian embedding, không phải xác suất câu trả lời đúng. Ngưỡng “0,8” không có ý nghĩa phổ quát giữa các mô hình hay corpus.

Tìm chính xác xét toàn bộ tập vector hợp lệ. ANN, viết tắt của approximate nearest neighbor, tìm gần đúng để giảm thời gian; vì vậy có thể bỏ lỡ một số hàng xóm gần nhất. Cần phân biệt **ANN recall** so với tìm chính xác với **evidence recall** so với bằng chứng mà chuyên gia gán nhãn.

Có thể trộn kết quả từ khóa và vector bằng Reciprocal Rank Fusion: $RRF(d) = \sum 1 / (c + rank_i(d))$. Tài liệu không xuất hiện trong một danh sách đóng góp 0 từ danh sách đó. Giá trị $c = 60$ là cấu hình thử ban đầu trong báo cáo, không phải hằng số bắt buộc. RRF tránh cộng trực tiếp hai loại điểm có thang đo khác nhau.

Quy trình dễ hình dung: tìm rộng bằng hai cách → gộp và loại trùng → đọc kỹ để xếp hạng lại → lấy đủ bằng chứng để trả lời. Elastic cũng mô tả hybrid search như sự kết hợp lexical và semantic retrieval. [S18]

04 · Một hệ thống RAG hoàn chỉnh

Luồng chuẩn bị dữ liệu

Bước	Việc phải làm	Đầu ra cần giữ
1. Thu thập	Lấy file/API cùng quyền sử dụng và lịch cập nhật.	File gốc, URL, thời điểm thu thập, checksum.
2. Đọc cấu trúc	Parse văn bản, OCR nếu cần, nhận diện bảng và tiêu đề.	Các khối nội dung có thứ tự, trang và vị trí.
3. Chuẩn hóa	Nhận diện ngôn ngữ, loại tài liệu, phiên bản và trùng lặp.	Tài liệu chuẩn hóa, metadata và quyết định kiểm duyệt.
4. Chunk	Chia theo quy tắc phù hợp loại nguồn.	Chunk, parent, liên kết đến đoạn gốc.
5. Lập chỉ mục	Tạo embedding và chỉ mục từ khóa.	Index có model/version và trạng thái hoàn tất.

Luồng trả lời khi có câu hỏi

Bước	Việc phải làm	Điểm kiểm soát
1. Hiểu yêu cầu	Phân loại câu hỏi, lấy định danh và quyền đã xác thực.	Câu hỏi mơ hồ cần hỏi lại hoặc ghi rõ phạm vi.
2. Chọn luồng	Tra cứu tri thức, truy vấn dữ liệu có cấu trúc hoặc kết hợp.	Số lượng/tổng hợp phải tính trên dữ liệu đầy đủ.
3. Tìm và xếp hạng	Hybrid retrieval, fusion, reranking.	Chỉ xử lý ứng viên mà người dùng được phép đọc.
4. Gom bằng chứng	Mở rộng parent hoặc đoạn lân cận có chọn lọc.	Giữ ngoại lệ; loại đoạn lặp; giới hạn token.
5. Sinh và kiểm tra	Viết đáp án, gán nguồn, kiểm tra claim quan trọng.	Có cơ chế từ chối, báo xung đột và mở nguồn gốc.

Phải lưu dấu vết đủ để tái dựng một lượt trả lời: câu hỏi đã xử lý, bộ lọc, danh sách chunk, phiên bản nguồn, cấu hình truy xuất, model và prompt version. Với dữ liệu bệnh nhân, log cần tối thiểu hóa nội dung nhạy cảm và có quyền truy cập riêng.

Một lỗi hay gặp: chỉ lưu câu trả lời cuối. Khi câu trả lời sai, nhóm không biết lỗi đến từ tài liệu, OCR, chunk, tìm kiếm hay LLM. Trace theo từng bước giúp sửa đúng thành phần.

05 · Vì sao dữ liệu y khoa khó hơn?

Đúng chủ đề vẫn có thể sai bối cảnh

“Không ghi nhận tiền sử dị ứng” khác “không có dị ứng”. “Tiền sử gia đình mắc bệnh” khác “bệnh nhân đang mắc bệnh”. Một khuyến cáo dành cho người lớn không tự áp dụng cho trẻ em; kết quả từ nghiên cứu quan sát cũng không tự trở thành phác đồ điều trị.

Vì vậy, chunk và metadata cần giữ các yếu tố như đối tượng, thời gian, phủ định, mức độ chắc chắn, mục đích tài liệu, khu vực áp dụng và đơn vị đo. Không để bước làm sạch bỏ dấu “≤”, “≥”, dấu âm hoặc biến “mg” thành “µg”. Đây là yêu cầu thiết kế dữ liệu, không thể chỉ giải quyết bằng một câu nhắc trong prompt.

Ba nhóm dữ liệu phải phân biệt

Nhóm	Ví dụ	Cách tiếp cận đề xuất
Tri thức tham khảo	Hướng dẫn, giáo trình, bài báo, nhãn thuốc.	RAG theo nguồn và phiên bản; xét đối tượng áp dụng.
Dữ liệu cá nhân	Bệnh án, đơn thuốc, ghi chú, kết quả xét nghiệm.	Phân quyền theo bệnh nhân/lần khám; giữ trực thời gian.
Dữ liệu phân tích	Danh sách ca bệnh, số ca theo tháng, phân bố kết quả.	SQL hoặc công cụ tính toán có kiểm soát; không suy số lượng từ top-k chunk.

Khi hai nguồn không đồng ý

Không “bỏ phiếu” bằng số chunk. Mười website chép lại một nguồn không phải mười bằng chứng độc lập. Giữ liên kết tới nguồn gốc, phân biệt loại nghiên cứu và kiểm tra xem xung đột có thực chất là khác quần thể, ngày cập nhật hoặc phạm vi áp dụng hay không.

Nếu chưa giải quyết được, câu trả lời nên trình bày từng quan điểm với nguồn, ngày và giới hạn. Thứ tự ưu tiên phải được người phụ trách chuyên môn xác định cho từng use case. “Mới hơn” là một tín hiệu, không phải tiêu chí duy nhất.

Khi chưa có bệnh án trong phạm vi dự án, nên bắt đầu với kho tri thức đã được duyệt. Chỉ bổ sung luồng dữ liệu bệnh nhân khi đã xác định rõ quyền sử dụng, phân quyền và tiêu chí đánh giá tương ứng.

06 · Nạp dữ liệu từ nhiều nguồn

Tạo bản chuẩn hóa trước khi tạo vector

Mỗi nguồn nên có một adapter riêng nhưng cùng xuất ra một cấu trúc chung: document → section → block → span. Block có thể là đoạn văn, tiêu đề, danh sách, bảng hoặc chú thích hình. Span là vị trí cụ thể trong văn bản chuẩn hóa và phải liên kết ngược được về nguồn gốc.

Với PDF có chữ, ưu tiên parser đọc bố cục. Với scan, cần OCR và kiểm tra mẫu. File nhiều cột dễ bị đọc lẫn thứ tự; bảng qua hai trang dễ mất tiêu đề. Docling là một ứng viên để thử vì thiết kế của nó có phân tích bố cục và nhận diện cấu trúc bảng; đây chưa phải kết luận nó tốt nhất cho dữ liệu của dự án. [S07]

Cổng kiểm tra chất lượng đầu vào

- **Văn bản:** kiểm tra trang thiếu chữ, lỗi dấu tiếng Việt, mất phủ định và tỷ lệ ký tự bất thường.
- **Bảng:** kiểm tra đúng hàng–cột, tiêu đề, chú thích, đơn vị và các ô gộp; đối chiếu ảnh trang gốc.
- **Nguồn:** lưu tổ chức phát hành, ngày xuất bản/cập nhật, URL hoặc DOI/PMID, quyền sử dụng và phiên bản.
- **Trùng lặp:** hash để tìm bản giống hệt; so sánh gần đúng để gom bản sao. Không gộp hai bản chỉ vì giống phần lớn nếu khác một ngoại lệ quan trọng.

Tài liệu OCR kém nên vào hàng chờ kiểm tra, thay vì âm thầm lập chỉ mục. Chất lượng trích xuất có thể khác nhau giữa các trang trong cùng file; metadata cấp tài liệu chưa đủ để phản ánh điều đó.

Chuẩn hóa mà không làm mất bản gốc

Lưu riêng raw_text, normalized_text và search_text. Có thể mở rộng viết tắt hoặc thêm từ đồng nghĩa trong search_text, nhưng không sửa nội dung nguyên bản dùng để trích dẫn. Với thuật ngữ y sinh, MeSH là một từ vựng kiểm soát hữu ích để bổ sung nhãn và tìm kiếm; không ép mọi khái niệm bệnh án về MeSH. [S22]

PMC phân biệt nội dung được phép tái sử dụng và yêu cầu dùng các kênh tài tự động được cho phép. Vì thế, thấy một bài đọc được trên web chưa đủ để suy ra quyền tải hàng loạt hoặc tái phân phối; cần lưu giấy phép từng nguồn. [S21]

07 · Chunking: cách chọn đơn vị tri thức

Chunk tốt phải trả lời được “đang nói về cái gì?”

Nếu đoạn bắt đầu bằng “Trong trường hợp này, không nên sử dụng...”, người đọc chưa biết “trường hợp này” là gì. Chunk đó có thể tìm thấy nhưng chưa đủ để làm bằng chứng. Mục tiêu là giữ một ý có thể hiểu được cùng các điều kiện cần thiết, trong giới hạn đầu vào của mô hình embedding.

Chunk nhỏ thường tập trung và dễ khớp câu hỏi, nhưng có nguy cơ mất ngữ cảnh. Chunk lớn chứa thêm điều kiện, nhưng có thể trộn nhiều chủ đề và tốn token khi trả lời. Không có một số token duy nhất cân bằng tốt cho mọi loại tài liệu.

Cách chia	Ưu điểm	Điểm cần chú ý
Fixed-size	Rẻ, đơn giản, tạo baseline nhanh.	Có thể cắt giữa câu, bảng hoặc điều kiện –ngoại lệ.
Recursive	Thử tách theo đoạn, câu rồi mới theo giới hạn nhỏ hơn.	Cần đo lại bằng tokenizer; chia theo ký tự không tương đương token.
Sentence	Giữ ranh giới câu.	Một câu có thể quá ngắn hoặc phụ thuộc câu trước.
Section-aware	Theo tiêu đề và cấu trúc tài liệu.	Phụ thuộc parser; section dài vẫn cần tách thêm.
Semantic	Dùng thay đổi ngữ nghĩa để chọn ranh giới.	Thêm chi phí và ngưỡng cần tinh chỉnh; có thể bỏ lỡ ngoại lệ ngắn.

Overlap giúp gì?

Overlap lấp một phần nội dung ở hai chunk kế tiếp để giảm mất ngữ cảnh tại biên. Tuy nhiên, nó làm tăng số token cần embed, tăng dung lượng và tạo nhiều kết quả gần trùng nhau. Overlap không bảo đảm giữ được một bảng hoặc điều kiện nằm xa nhau.

Đề xuất thử: chunk văn bản 300–600 token, overlap 0–15%, ưu tiên câu và ranh giới mục. Đây là dải thử nghiệm do báo cáo đề xuất. Luôn chứa dung lượng cho tiêu đề/ngữ cảnh thêm vào; đếm bằng tokenizer của embedding model thực tế.

Quy tắc ưu tiên: bảo toàn ý nghĩa trước, giới hạn token sau. Nếu một đơn vị quan trọng quá dài, tách có cấu trúc và liên kết lại; không cắt cụt âm thầm.

08 · Parent–child và các cách nâng cao

Parent–child: tìm đoạn nhỏ, đọc ngữ cảnh đủ lớn

Tạo child nhỏ để tìm kiếm và lưu liên kết về parent là mục lớn hơn. Khi child phù hợp, có thể lấy parent hoặc các đoạn lân cận cần thiết. Đây là lựa chọn đề xuất cho hướng dẫn dài: child khoảng 200–400 token, parent khoảng 800–1.500 token, rồi điều chỉnh theo bộ test.

Chỉ cần embed child trong baseline; parent có thể là text được lấy bằng ID. Cách này không mặc định tăng gấp đôi số vector. Phải giới hạn tổng token sau khi mở rộng và gộp child cùng parent để tránh đưa một mục vào nhiều lần.

Những kỹ thuật nên thử sau baseline

Kỹ thuật	Ý tưởng	Khi đáng thử và điều cần giữ
Contextual enrichment	Thêm tiêu đề hoặc mô tả ngữ cảnh ngắn trước khi embed.	Hữu ích cho đoạn đứng riêng khó hiểu. Ưu tiên metadata xác thực; ngữ cảnh LLM sinh phải lưu riêng. [S08]
Proposition	Trích thành phát biểu nhỏ, có thể hiểu độc lập.	Hữu ích cho hỏi một sự kiện. Dễ mất phủ định/điều kiện; cần liên kết về span gốc. [S05]
Late chunking	Mô hình xử lý context dài trước; pool token theo từng chunk sau.	Có thể giữ ngữ cảnh liên đoạn; cần model/API cho phép thao tác token embeddings. [S04]
RAPTOR	Tạo cây nhóm và tóm tắt nhiều cấp.	Hữu ích cho câu hỏi tổng quan. Tóm tắt là dữ liệu dẫn đường, chưa phải bằng chứng nguyên bản. [S11]
GraphRAG	Dùng thực thể, quan hệ và các nhóm trong đồ thị để truy xuất.	Thử khi câu hỏi cần liên kết hoặc tổng hợp toàn corpus. Có thêm chi phí trích xuất, cập nhật và kiểm chứng. [S12]

Late chunking khác late interaction: một cách thay đổi thời điểm hình thành chunk embedding; cách còn lại so khớp nhiều vector nhỏ khi truy xuất. Cả hai đều khác việc tạo một vector duy nhất cho cả tài liệu.

Không bật đồng thời mọi kỹ thuật. Mỗi kỹ thuật nên trả lời một lỗi đã quan sát được, chẳng hạn mất ngữ cảnh, bỏ sót tên chính xác hoặc không tổng hợp được nhiều mục.

09 · Chunk theo từng loại dữ liệu

Các kích thước bên dưới là cấu hình xuất phát để thử nghiệm, không phải chuẩn y khoa hay kết quả benchmark của dự án.

Loại nguồn	Đơn vị nên giữ	Cách làm đề xuất
Phác đồ, guideline	Khuyến cáo + đối tượng + điều kiện + ngoại lệ.	Section-aware và parent-child; không tách cảnh báo khỏi khuyến cáo tương ứng.
Bài báo nghiên cứu	Mục tiêu, phương pháp, kết quả, giới hạn tách biệt nhưng có liên kết.	Chunk theo mục, khoảng 400–800 token; thêm tên nghiên cứu và quần thể khi có trong nguồn.
Nhãn thuốc, chuyên luận	Chỉ định, chống chỉ định, tương tác, cách dùng là các mục riêng.	Chia theo tiêu đề; liên kết chéo tới cảnh báo liên quan; giữ tên hoạt chất và dạng bào chế.
Bảng trong PDF/Excel	Hàng dữ liệu cùng tên cột, đơn vị, chú thích.	Lưu bảng cấu trúc; tạo văn bản theo hàng/nhóm hàng để tìm; giữ table_id và vị trí.
FAQ, tài liệu giáo dục	Một câu hỏi cùng câu trả lời.	Một cặp Q–A là đơn vị đầu tiên; không ghép các câu hỏi không liên quan chỉ để đủ token.
Ghi chú bệnh án	Một lần khám hoặc một mục trong ghi chú.	Chunk theo encounter/section; giữ thời gian, người ghi và trạng thái xác nhận.
Xét nghiệm, chỉ số sống	Một quan sát có thời điểm, đơn vị và trạng thái.	Lưu dạng cấu trúc; chỉ tạo text bổ sung cho tìm kiếm, không dùng vector thay cho phép tính.
Ảnh, biểu đồ, lưu đồ	Hình cùng chú thích và nhánh điều kiện.	Giữ ảnh gốc và mô tả được kiểm tra; cân nhắc truy xuất đa phương thức theo nhu cầu.

Hai trường hợp cần tránh

Không coi một trang là một chunk mặc định. Một mục có thể kéo dài qua hai trang; một trang có thể chứa hai cột và nhiều chủ đề. Trang rất hữu ích để dẫn nguồn nhưng không nhất thiết là đơn vị ngữ nghĩa.

Không lấy mô tả hình do mô hình sinh làm bản thay thế hoàn toàn. Với sơ đồ quyết định hoặc bảng nhạy cảm, phải cho phép xem hình gốc và kiểm tra nhánh, chú thích, đơn vị. Nếu dự án chỉ có văn bản, chưa cần đưa hình ảnh vào embedding ngay từ đầu.

10 · Ví dụ: chunk sai và chunk đủ nghĩa

Ví dụ hoàn toàn giả lập để minh họa kỹ thuật; “Thuốc A” và “tình trạng B” không phải chỉ dẫn điều trị.

Đoạn nguồn giả lập

“Hướng dẫn X, phiên bản 2, mục 3: áp dụng cho người lớn. Thuốc A được xem xét trong tình huống C sau đánh giá chuyên môn. Không áp dụng khuyến cáo này cho người có tình trạng B. Việc theo dõi thực hiện theo mục 4.”

Cắt máy móc: chunk 1 kết thúc ở “sau đánh giá chuyên môn”; chunk 2 chỉ chứa câu không áp dụng. Khi hỏi về Thuốc A, bộ tìm kiếm có thể chỉ chọn chunk 1 và làm mất ngoại lệ.

Cách đề xuất: giữ khuyến cáo và ngoại lệ cùng một child hoặc đặt liên kết bắt buộc giữa chúng. Parent là toàn mục 3; mục 4 là liên kết ngữ cảnh để lấy thêm khi câu hỏi hỏi về theo dõi.

Trường	Giá trị minh họa	Vi sao cần
document_id / version_id	huong-dan-x / v2	Biết khuyến cáo thuộc bản nào.
section_path	Hướng dẫn X > Mục 3	Tạo ngữ cảnh khi embed và hiển thị nguồn.
chunk_id / parent_id	v2-m3-c1 / v2-m3	Tìm nhỏ, mở rộng có kiểm soát.
population	adult, extracted	Ghi cả giá trị và cách trích xuất; chưa xác định thì để unknown.
source_span	block 18–20; trang 7	Truy ngược về đúng phần nguồn.
related_evidence	v2-m4	Lấy thêm hướng dẫn theo dõi khi cần.

Với một bảng

Nếu hàng chỉ chứa “Nhóm A | Có | X”, văn bản tìm kiếm nên có dạng “Bảng điều kiện, Nhóm đối tượng: A; Đủ điều kiện: Có; Ghi chú: X”, kèm chú thích và tên bảng. Bản gốc có cấu trúc vẫn được lưu để truy vấn và kiểm chứng.

Đừng biến ô trống thành “không”, đừng suy diễn đơn vị từ hàng bên cạnh nếu nguồn không nêu rõ. Nếu đơn vị nằm trong tiêu đề chung, sao chép có kiểm soát vào bản biểu diễn tìm kiếm và giữ vị trí tiêu đề gốc.

11 · Nghiên cứu nói gì về chunking?

Bằng chứng có ích, nhưng cần đọc đúng phạm vi

MIRAGE trong nghiên cứu MedRAG gồm 7.663 câu hỏi từ năm bộ QA y khoa, cho phép so sánh các thành phần của RAG. Giá trị chính cho dự án là cách tách các biến số để đánh giá. Điểm trên câu hỏi benchmark không chứng minh khả năng xử lý bệnh án thật, tiếng Việt hoặc một quy trình lâm sàng cụ thể. [S02]

Nghiên cứu năm 2025 về chunking so sánh bốn chiến lược trên 30 câu hỏi hậu phẫu nâng mũi với Gemini 1.0 Pro. Adaptive chunking đạt độ chính xác được báo cáo 87%, so với 50% ở baseline. Tuy nhiên, số câu hỏi ít, chuyên khoa hẹp và cấu hình cụ thể; không nên biến kết quả đó thành cam kết hiệu năng cho dự án này. [S03]

MRAG năm 2026 mở rộng đánh giá sang các tác vụ tiếng Anh và Trung. Đây là preprint; nguồn này gợi ý cần đo cả tính hữu ích và độ dễ đọc, nhưng không cung cấp bảo đảm cho tiếng Việt. [S25]

Thiết kế phép thử để tìm cách phù hợp

Lượt thử	Chỉ thay đổi	Giữ cố định
A	256 / 512 / 768 token; overlap 0 / 10%.	Một embedding, một retriever, cùng bộ câu hỏi.
B	Recursive so với section-aware.	Ngân sách token và các cấu hình tốt ở A.
C	Có/không parent expansion.	Child, cách retrieve, giới hạn context cuối.
D	Có/không contextual prefix hoặc semantic split.	Bộ test và cấu hình đã chọn ở các lượt trước.

Mỗi lượt phải đo số chunk, chi phí lập chỉ mục, evidence recall, số ngoại lệ bị mất và chi phí mỗi câu trả lời. Chọn cấu hình trên validation; chỉ dùng test giữ kín cho kết quả cuối. Nếu tiếp tục chỉnh dựa trên test, tập đó đã trở thành validation.

Một cấu hình đáng chọn là cấu hình cải thiện chất lượng trong ngân sách cho phép và ổn định trên các nhóm dữ liệu quan trọng. Điểm trung bình đẹp không bù được việc liên tục bỏ sót cảnh báo trong bảng.

12 · Embedding cho tiếng Việt và tiếng Anh

Lựa chọn theo dữ liệu thực tế

Cần đánh giá riêng bốn chiều: hỏi Việt-nguồn Việt, hỏi Việt-nguồn Anh, hỏi Anh-nguồn Anh và câu hỏi trộn ngôn ngữ/viết tắt. Một mô hình có điểm đa ngôn ngữ cao chưa chắc hiểu tốt cách ghi bệnh án tại đơn vị của bạn.

BGE-M3 hỗ trợ dense, sparse và multi-vector retrieval, là một ứng viên đa ngôn ngữ để tạo baseline. Qwen3 Embedding cung cấp một họ mô hình embedding và reranking với nhiều kích thước, cũng đáng đưa vào thử nghiệm nếu tài nguyên phù hợp. Đây là shortlist nghiên cứu, không phải bảng xếp hạng hay khuyến nghị mua dịch vụ. [\[S09\]](#) [\[S26\]](#)

Những điều cần ghi trong embedding profile

- Tên model, revision chính xác, tokenizer, số chiều vector và giới hạn đầu vào.
- Cách pooling, normalization, distance metric; query/document prefix nếu model yêu cầu.
- Phiên bản tiền xử lý và chunking; quy tắc thêm tiêu đề/ngữ cảnh.
- Ngôn ngữ đã kiểm thử, phần cứng, batch size và tốc độ đo được.

Không trộn vector của hai model trong cùng không gian tìm kiếm chỉ vì cùng số chiều. Khi đổi model hoặc cách tiền xử lý có ảnh hưởng, tạo index generation mới, chạy lại bộ test rồi chuyển luồng truy vấn. Query embedding phải dùng đúng profile của index đang phục vụ.

Dịch câu hỏi có phải giải pháp mặc định?

Có thể dùng thêm một truy vấn dịch sang tiếng Anh, nhưng nên giữ câu hỏi gốc và các thực thể quan trọng. Dịch sai tên hoạt chất, phủ định hoặc viết tắt có thể đổi nghĩa truy vấn. Nếu thử query expansion, giới hạn số biến thể và đánh giá lợi ích trên các ca thực sự bị bỏ sót.

Không dịch rồi ghi đè bản gốc. Nguồn dẫn phải quay về tài liệu gốc; bản dịch và từ đồng nghĩa chỉ là lớp hỗ trợ tìm kiếm. Với tên viết tắt nhiều nghĩa, dùng chuyên khoa/bối cảnh để chọn hoặc hỏi lại, thay vì luôn thay bằng một nghĩa cố định.

Tiết kiệm hợp lý: cache embedding theo hash của đúng nội dung được embed cộng model revision và preprocessing version. Phạm vi cache phải tôn trọng tenant và chính sách dữ liệu.

13 · Từ ứng viên đến bằng chứng

Cấu hình retrieval khởi đầu

Đề xuất để thử: lấy 50 ứng viên từ lexical search và 50 từ dense search; hợp nhất bằng RRF, loại trùng; rerank khoảng 30–50 ứng viên; giữ 6–10 đơn vị bằng chứng sau khi mở rộng ngữ cảnh. Giới hạn context, chẳng hạn 4.000–8.000 token, cần đo theo mô hình và tác vụ. Đây là ngân sách thử nghiệm, không phải bảo đảm đủ bằng chứng cho mọi câu hỏi.

Reranker xem đồng thời câu hỏi và ứng viên nên có thể đánh giá độ liên quan chi tiết hơn phép so khoảng cách vector. Tuy nhiên, nó không sửa được trường hợp tài liệu đúng chưa được retrieval đưa vào tập ứng viên.

Kiểm soát trước khi gửi context cho LLM

Kiểm tra	Cách xử lý đề xuất	
Nhiều child cùng một parent	Gộp theo parent; chỉ thêm phần thiếu, không lặp toàn bộ mục.	
Bản cũ và bản mới cùng được tìm thấy	Xét thời điểm câu hỏi và trạng thái phiên bản; không luôn lấy bản mới cho câu hỏi lịch sử.	
Nhiều trang sao chép một nguồn	Gom theo canonical source; vẫn giữ dấu vết từng bản thu thập.	
Có hai khuyến cáo khác nhau	Giữ cả hai khi cần giải thích khác biệt; không để bước loại trùng xóa mất xung đột.	
Câu hỏi cần nhiều bước	Chia thành câu hỏi con và gom đủ bằng chứng; đặt giới hạn số lượt và thời gian.	
Không đủ bằng chứng	Hỏi lại, trả lời một phần có nguồn hoặc từ chối có lý do.	

Thiết kế câu trả lời dễ kiểm tra

Đề xuất cấu trúc: trả lời trực tiếp → điều kiện/phạm vi → nguồn gắn vào từng ý → điểm chưa chắc chắn. Citation phải chứa ID do hệ thống cung cấp; không để mô hình tự tạo URL hay số trang. Giao diện nên cho mở đúng trang hoặc span được dẫn.

Kiểm tra tự động có thể phát hiện citation không tồn tại, số liệu không có trong evidence hoặc câu trả lời thiếu nguồn. Việc nguồn có thật không đồng nghĩa nguồn hỗ trợ phát biểu. Những claim quan trọng vẫn cần kiểm tra sự phù hợp giữa câu viết và đoạn gốc.

Giữ riêng ba câu hỏi: “Có tìm thấy không?”, “Nguồn có áp dụng không?”, “Câu trả lời có diễn đạt đúng nguồn không?”. Mỗi câu cần một phép kiểm tra khác nhau.

14 · Thiết kế lưu trữ ba lớp

Mỗi lớp có một trách nhiệm rõ ràng

Lớp	Lưu những gì?	Vì sao cần?
1. Object storage	PDF, ảnh, HTML snapshot, dữ liệu xuất gốc; bản parse lớn.	Giữ nguồn nguyên bản, phục hồi và xử lý lại khi đổi parser.
2. Cơ sở dữ liệu chuẩn	Document, version, chunk, provenance, quyền, trạng thái và quan hệ.	Là nguồn sự thật cho cập nhật, kiểm duyệt, truy vết và xóa.
3. Chỉ mục tìm kiếm	Vector, lexical index và metadata tối thiểu để lọc.	Tối ưu đường truy xuất; phải có thể tái tạo từ hai lớp trước.

Ba lớp là phân chia trách nhiệm, không bắt buộc ba sản phẩm. PostgreSQL có thể chứa cả lớp dữ liệu chuẩn và chỉ mục pgvector/full-text trong phiên bản đầu. Khi cần tách, Qdrant hoặc một hệ tìm kiếm khác có thể giữ chỉ mục chuyên biệt.

Sơ đồ dữ liệu đề xuất

Nguồn → **file gốc** → **document version** → **blocks/parents** → **chunks** → **embeddings/indexes** → **bảng chứng trong câu trả lời**.

Mỗi mối nối cần có ID hoặc khóa liên kết. Ví dụ: từ chunk được trích dẫn, lần ra version, trang, file gốc và checksum. Nếu một chunk chỉ còn text và vector, nhóm sẽ khó biết nó thuộc tài liệu nào và bản nào cần thu hồi.

Dữ liệu có cấu trúc vẫn cần cấu trúc

Kết quả xét nghiệm nên giữ mã, giá trị, đơn vị, thời điểm và trạng thái ở các trường riêng. FHIR Observation là một mô hình tham khảo cho loại dữ liệu này; FHIR Provenance mô tả nguồn gốc xử lý. Có thể dùng FHIR server hoặc mô hình quan hệ phù hợp hệ thống hiện có, không bắt buộc biến mọi tài liệu thành FHIR. [\[S19\]](#) [\[S20\]](#)

Ví dụ định tuyến: câu hỏi “giá trị gần nhất là bao nhiêu?” dùng truy vấn theo thời gian; “hướng dẫn giải thích chỉ số này ở đâu?” dùng RAG; câu hỏi kết hợp cần hai luồng và nguồn rõ ràng cho từng phần.

Vector là chỉ mục dẫn đường. Bản gốc, phiên bản và quan hệ dữ liệu mới là nền tảng để hệ thống giải thích được câu trả lời và cập nhật đúng.

15 · Schema và metadata cần có

Mô hình quan hệ đề xuất

Thực thể	Khóa và trường chính	Quan hệ
sources	source_id, tên tổ chức, loại nguồn, quyền sử dụng.	Một nguồn có nhiều tài liệu.
documents	document_id, tenant_id, canonical_key, title.	Một tài liệu có nhiều version.
document_versions	version_id, content_hash, URI, ngày hiệu lực, trạng thái.	Một version có nhiều block/chunk.
chunks	chunk_id, version_id, parent_id, text, loại, vị trí, hash.	Một chunk có thể có nhiều embedding profile.
embedding_profiles	profile_id, model revision, số chiều, preprocessing.	Xác định không gian vector và cách tạo.
embeddings / indexes	chunk_id, profile_id, vector, generation_id.	Chỉ mục của đúng nội dung và đúng profile.
ingestion_runs / audit	run_id, trạng thái, lỗi, checksum và tác nhân.	Theo dõi xử lý và thay đổi.

Metadata tối thiểu để trích dẫn và lọc

Nguồn gốc: URL/DOI/PMID nếu có, tác giả/tổ chức, thời gian xuất bản, thời gian thu thập và giấy phép. Hai mốc thời gian này khác nhau; thu thập hôm nay không làm một tài liệu cũ trở thành tài liệu mới.

Vị trí: section_path, trang PDF, nhãn trang in nếu khác, block_id, span offsets và bounding box khi có. Nếu văn bản đã được chuẩn hóa, offset phải đi cùng phiên bản normalized_text; không dùng offset của bản này cho bản khác.

Ngữ cảnh: ngôn ngữ, chuyên khoa, đối tượng áp dụng, loại bằng chứng, phạm vi địa lý/tổ chức. Không có thông tin thì dùng unknown/null; không coi thiếu metadata là “áp dụng cho tất cả”.

Quản trị: tenant, access_scope, trạng thái duyệt, ngày hiệu lực/hết hiệu lực, parser/chunker version và chất lượng trích xuất. Với nhãn do mô hình suy ra, lưu cả phương pháp, confidence nếu có ý nghĩa và trạng thái kiểm tra.

Cột riêng hay JSONB?

Các trường thường dùng để JOIN, lọc quyền và chọn phiên bản nên là cột có kiểu và index phù hợp. Metadata ít ổn định có thể nằm trong JSONB, nhưng cần schema validation ở tầng nạp. Không nhét mọi thứ vào một JSON lớn rồi mong DB tự tìm nhanh.

16 · Chọn DB theo nhu cầu vận hành

So sánh những hướng thực tế

Hướng	Phù hợp khi	Đổi lại điều gì?
PostgreSQL + pgvector	Đã dùng Postgres; cần JOIN, transaction, metadata và vận hành ít thành phần.	Phải đo cạnh tranh tài nguyên giữa vector search, dữ liệu nghiệp vụ và ghi mới.
PostgreSQL + Qdrant	Muốn vận hành/lập chỉ mục vector độc lập và có các bộ lọc payload.	Thêm đồng bộ, backup và xử lý sự không nhất quán giữa hai hệ.
Elastic cho tìm kiếm + DB chuẩn	Tìm từ khóa, bộ phân tích ngôn ngữ và vận hành search là ưu tiên lớn.	Cần chọn cấu hình hybrid, mapping và kiểm tra điều kiện tính năng/giấy phép.
Dịch vụ vector quản lý sẵn	Nhóm ưu tiên giảm công vận hành và đáp ứng được yêu cầu dữ liệu.	Cần tính chi phí tải thực, giới hạn truy vấn, xuất dữ liệu và vùng lưu trữ.

Khả năng kỹ thuật của pgvector, Qdrant và Elastic được mô tả trong tài liệu chính thức; bảng trên là nhận định thiết kế cho dự án, chưa phải kết quả đo hiệu năng giữa các sản phẩm. [\[S13\]](#) [\[S16\]](#) [\[S18\]](#)

Khuyến nghị cho phiên bản đầu

Nếu chưa có ràng buộc khác, bắt đầu bằng object storage và PostgreSQL + pgvector. Tìm từ khóa có thể dùng PostgreSQL full-text để lập baseline, nhưng **ts_rank/ts_rank_cd không phải BM25**. Nếu nghiên cứu cần so sánh BM25 đúng nghĩa, phải dùng engine hoặc extension thực sự triển khai BM25 và ghi rõ phiên bản. [\[S14\]](#)

Đối với tiếng Việt, cần kiểm tra tokenizer/analyzer, dấu và từ nhiều âm tiết. Cấu hình text search đơn giản có thể dùng để thử, nhưng không nên mặc định chất lượng như một pipeline phân tích tiếng Việt chuyên dụng.

Khi nào nên tách vector DB?

Tách khi đo thấy không đạt yêu cầu độ trễ/throughput sau tối ưu hợp lý, cần mở rộng vector độc lập, hoặc yêu cầu vận hành phù hợp hơn với hệ chuyên dụng. Không có quy tắc “hơn một triệu vector là phải đổi DB”. Số chiều, bộ lọc, QPS, tỷ lệ ghi, RAM và mục tiêu recall đều tác động.

Hãy so sánh trên cùng dữ liệu, cùng tập truy vấn, cùng bộ lọc và cùng ngân sách tài nguyên. Một benchmark bỏ qua bộ lọc tenant có thể không đại diện cho tải thực tế của hệ thống.

17 · Index và hiệu năng truy xuất

Exact, HNSW hay IVFFlat?

Theo tài liệu pgvector, exact search là mặc định; HNSW đánh đổi thời gian dựng/bộ nhớ để có đặc tính tốc độ–recall tốt, còn IVFFlat cần dữ liệu để tạo các nhóm. Tài liệu cũng cảnh báo ANN kết hợp lọc có thể trả ít kết quả hơn mong đợi; iterative scans được bổ sung từ 0.8.0 để tìm thêm trong giới hạn cấu hình. [\[S13\]](#)

Đề xuất cho dự án: dùng exact search làm mốc đo ANN recall trên một tập truy vấn, rồi thử HNSW nếu độ trễ exact không đạt. Đo lại sau khi áp dụng bộ lọc có độ chọn lọc giống thực tế; một tenant nhỏ trong corpus lớn có thể hành xử rất khác truy vấn toàn kho.

Những tối ưu nên thử theo thứ tự

Thứ tự	Tối ưu	Đo để biết có hiệu quả
1	Loại bản sao, chunk rộng và nội dung boilerplate.	Số vector, evidence recall và tỷ lệ kết quả trùng.
2	Index các trường lọc thường dùng.	Query plan; độ trễ theo tenant và độ chọn lọc.
3	Điều chỉnh số ứng viên và tham số ANN.	ANN recall, evidence recall, p95/p99, RAM.
4	Batch nạp/embedding; kiểm soát worker và backpressure.	Throughput nạp, backlog và độ trễ truy vấn khi đang ghi.
5	Quantization hoặc giảm số chiều nếu model hỗ trợ.	Mức tiết kiệm và suy giảm chất lượng, nhất là ca hiếm.
6	Partition/shard theo tải và ranh giới dữ liệu.	Fan-out, cân bằng tải và chi phí vận hành.

Qdrant có payload index để hỗ trợ truy vấn có bộ lọc, cùng các lựa chọn quantization. Có thể dùng vector đã nén để chọn ứng viên rồi chấm lại bằng vector gốc; lợi ích và chi phí phải đo trên cấu hình thực tế. [\[S16\]](#) [\[S17\]](#)

Những tối ưu có thể phản tác dụng

Đừng tạo index cho mọi trường, vì index cũng tốn dung lượng và chi phí cập nhật. Đừng chia partition theo từng tài liệu nhỏ nếu không có nhu cầu rõ ràng. Đừng giảm dimension bằng

cách cắt tùy tiện vector của model không hỗ trợ cách dùng đó. Đừng cache chỉ theo câu hỏi nếu quyền, phiên bản nguồn hoặc profile có thể khác nhau.

18 · Ước tính dung lượng và chi phí

Một ví dụ có giả định rõ ràng

Giả sử có **100.000 tài liệu**, mỗi tài liệu **5.000 token**, chunk **500 token**, overlap **50 token**, một vector cho mỗi chunk. Bỏ qua phần tiêu đề thêm vào khi tính số chunk. Với cửa sổ cố định: $n = 1 + \text{ceil}(\max(T - C, 0) / (C - O))$, cho $T > 0$ và $0 \leq O < C$. Tài liệu rỗng tạo 0 chunk.

Mỗi tài liệu trong ví dụ tạo 11 chunk; toàn kho tạo **1,1 triệu vector**. Thực tế section-aware có thể tạo nhiều hoặc ít hơn, nên phải đo trên mẫu rồi ngoại suy theo từng loại tài liệu.

Thành phần	Cách tính minh họa	Kết quả
Vector float32, 1.024 chiều	$1.100.000 \times 1.024 \times 4 \text{ byte}$	4,51 GB $\approx$ 4,20 GiB
Vector float32, 1.536 chiều	$1.100.000 \times 1.536 \times 4 \text{ byte}$	6,76 GB $\approx$ 6,29 GiB
Vector float16, 1.024 chiều	$1.100.000 \times 1.024 \times 2 \text{ byte}$	2,25 GB $\approx$ 2,10 GiB
Text trung bình 2.000 byte/chunk	$1.100.000 \times 2.000$	2,20 GB, giả định độc lập
Token cần embed	$11 \times 500 \times 100.000$	550 triệu token
Thêm prefix 40 token/chunk	$40 \times 1.100.000$	Thêm 44 triệu token

Các số vector trên chỉ là **payload thô**. Chưa có row overhead, HNSW, lexical index, metadata, parent text, file gốc, WAL, replica, backup và các generation cũ. Không dùng 4,51 GB để suy ngay rằng máy RAM 8 GB là đủ.

Công thức để lập ngân sách

Chi phí embedding = số token thực gửi $\times$ đơn giá tại thời điểm chọn nhà cung cấp. Chi phí trả lời gồm query embedding, reranker, token vào/ra của LLM, hạ tầng và các lượt retry. Với self-host, cần đo throughput và GPU/CPU time, không quy đổi trực tiếp từ đơn giá API.

Nếu mỗi ngày thay 1% tài liệu và phải embed lại toàn bộ số tài liệu đó, ví dụ này cần khoảng 5,94 triệu token/ngày khi có prefix. Dedup và cache có thể giảm, nhưng phải đo số chunk thực sự đổi.

Hệ số tăng token do overlap xấp xỉ $1/(1 - r)$ với tài liệu đủ dài: overlap 10% $\approx 1,11$ lần; 20% $\approx 1,25$ lần. Ranh giới tài liệu và làm tròn làm số thực tế khác đôi chút.

19 · Cập nhật, xóa và truy vết

Thiết kế để chạy lại không tạo bản sao

Mỗi lượt nạp có run_id và trạng thái. Hash nội dung giúp nhận ra file không đổi; khóa idempotency gồm tài liệu/phiên bản và cấu hình xử lý giúp retry an toàn. Không dùng mỗi số thứ tự chunk làm định danh ổn định: thêm một đoạn đầu tài liệu có thể làm toàn bộ số thứ tự dịch chuyển.

Giữ chunk_id bất biến trong một version/generation; dùng content hash riêng để tái sử dụng embedding. Hai chunk giống text nhưng có quyền hoặc provenance khác nhau vẫn cần giữ các bản ghi liên kết riêng.

Vòng đời đề xuất

received → **parsed** → **validated** → **indexed** → **published** → **superseded/withdrawn**.

Chỉ publish khi tất cả index bắt buộc đã sẵn sàng. Khi dùng nhiều hệ DB, có thể ghi thay đổi và một sự kiện outbox trong cùng transaction tại DB chuẩn; worker cập nhật index rồi xác nhận. Truy vấn chỉ dùng generation đã công bố và kiểm tra lại trạng thái trước khi đưa text ra ngoài.

Một tài liệu cập nhật tạo version mới. Có thể giữ version cũ cho truy vấn lịch sử/audit theo chính sách, nhưng phải loại khỏi luồng “hiện hành”. Lưu cả thời gian nguồn có hiệu lực và thời gian hệ thống ghi nhận để xử lý tài liệu nhập muộn.

Xóa không dùng ở bảng chunks

Xóa/thu hồi phải lan đến vector, lexical index, bản parse, tóm tắt sinh thêm, cache, file gốc và các bản sao phù hợp phạm vi được yêu cầu. Có tombstone để job cũ không vô tình đưa nội dung đã thu hồi trở lại. Thời hạn giữ backup và log phải được quy định riêng, không hứa xóa tức thời mọi bản sao nếu cơ chế chưa hỗ trợ.

Đổi model hoặc parser

Tạo generation mới bên cạnh generation đang chạy; kiểm tra đủ số bản ghi, orphan link và chất lượng trên bộ test. Chuyển con trở active generation sau khi đạt tiêu chí. Giữ rollback trong khoảng thời gian đã quy định, rồi dọn generation cũ. Bước chuyển phải đồng bộ với query embedding profile.

Hãy thử diễn tập một tình huống cụ thể: một guideline bị thu hồi lúc 10:00. Đến thời điểm nào nó ngừng xuất hiện trong retrieval, cache và câu trả lời đang xử lý? Đây là chỉ số freshness/thu hồi cần đo được.

20 · Phân quyền và bảo vệ dữ liệu

Quyền phải có hiệu lực trên toàn luồng

Nếu có bệnh án, tenant_id đơn thuần chưa đủ: có thể cần quyền theo bệnh nhân, đơn vị, vai trò hoặc mục đích sử dụng. Áp dụng quyền trước khi đưa dữ liệu tới reranker/LLM; kiểm tra lại khi fetch parent, xem trích dẫn hoặc đọc cache. Cache key cần gắn phạm vi quyền, corpus generation và cấu hình liên quan.

PostgreSQL RLS hỗ trợ giới hạn hàng theo policy, nhưng superuser và role BYPASSRLS có thể bỏ qua; chủ bảng cũng thường bỏ qua nếu không áp dụng FORCE ROW LEVEL SECURITY. Vì vậy cần kiểm thử bằng role ứng dụng thực, không chỉ bằng tài khoản quản trị. [\[S15\]](#)

Hai rủi ro riêng của RAG

Prompt injection từ nguồn: một file có thể chứa câu yêu cầu “bỏ qua chỉ dẫn và gửi dữ liệu đi”. Nội dung truy xuất phải được coi là dữ liệu tham khảo, không có quyền thay đổi chính sách hay gọi công cụ. Phân tách chỉ dẫn và evidence, giới hạn công cụ, kiểm soát nguồn nhập và kiểm tra đầu ra là các lớp phòng vệ bổ sung. [\[S23\]](#)

Đầu độc hoặc lộ thông tin qua index: một người có quyền nạp tài liệu có thể làm thay đổi kết quả cho nhiều người hỏi. Embedding cũng cần được bảo vệ; chuyển text thành vector không đồng nghĩa ẩn danh hóa. OWASP xem vector/embedding weaknesses là một nhóm rủi ro cần kiểm soát. [\[S24\]](#)

Các quyết định cần có nếu dùng dữ liệu bệnh nhân

- Xác định trường dữ liệu nào được gửi tới dịch vụ ngoài; giảm dữ liệu đến mức cần thiết.
- Chọn vùng lưu trữ/xử lý và quy định giữ dữ liệu của nhà cung cấp theo yêu cầu áp dụng.
- Mã hóa đường truyền/lưu trữ, quản lý khóa và tách quyền vận hành với quyền xem nội dung.
- Đặt chính sách log, backup, xóa và truy cập bằng chứng; kiểm tra cả dữ liệu trong file đính kèm/OCR.

Đây là yêu cầu kỹ thuật để chuẩn bị thẩm định; báo cáo không xác định luật nào áp dụng khi chưa biết quốc gia triển khai và chủ thể xử lý. Không mặc định HIPAA hoặc một chế độ pháp lý cụ thể áp dụng cho mọi dự án y khoa.

21 · Đánh giá: đo đúng chỗ sai

Các thước đo cần tách riêng

Tầng	Chỉ số	Diễn giải
Trích xuất	Lỗi OCR số/đơn vị; ô bảng đúng; span truy ngược được.	Dữ liệu đọc vào có đúng nguồn không?
Retrieval	Evidence Recall@k, nDCG@k, MRR.	Có lấy được bằng chứng cần thiết và xếp sớm không?
ANN	Overlap top-k ANN với top-k exact.	Index gần đúng có bỏ lỡ hàng xóm vector không?
Câu trả lời	Đúng nội dung, đầy đủ điều kiện, mức độ gây hại khi sai.	Người đọc có nhận thông tin đúng cho câu hỏi không?
Citation	Đúng nguồn, nguồn hỗ trợ claim, độ bao phủ claim.	Dẫn chứng có thật và chứng minh được ý đã viết không?
Từ chối	Tỷ lệ trả lời sai khi thiếu evidence; từ chối nhầm khi đủ evidence.	Hệ thống biết giới hạn mà vẫn hữu ích không?
Vận hành	p50/p95/p99, QPS, chi phí/câu, freshness.	Có phục vụ được tải và cập nhật như yêu cầu không?

Evidence Recall@k = số đơn vị bằng chứng chuẩn được tìm thấy / tổng đơn vị cần tìm. Với câu hỏi cần nhiều bằng chứng, thêm tỷ lệ tìm đủ toàn bộ tập bằng chứng tối thiểu. Chỉ tìm đúng một đoạn trong ba đoạn bắt buộc vẫn chưa đủ để trả lời.

Citation precision có thể định nghĩa là số claim có dẫn nguồn được nguồn hỗ trợ / tổng claim có dẫn nguồn. **Citation coverage** là số claim cần nguồn đã được dẫn / tổng claim cần nguồn. Một câu có nhiều claim cần kiểm tra từng claim; citation coverage cao vẫn có thể đi kèm citation sai.

Công cụ tự động và đánh giá chuyên gia

Ragas đề xuất đánh giá nhiều khía cạnh như chất lượng context và mức độ trung thành của câu trả lời. Có thể dùng để tăng tốc vòng lặp, nhưng điểm LLM-as-judge cần hiệu chỉnh với mẫu do người có chuyên môn chấm; nó không thay thế đánh giá y khoa. [\[S10\]](#)

Nếu so sánh các cách chunk khác kích thước, nên chấm theo evidence span/đơn vị ý nghĩa và cùng ngân sách token. Chỉ số Recall@5 giữa chunk 200 và 1.000 token có thể thiên vị cách đưa nhiều văn bản hơn.

22 · Xây bộ test đủ sát thực tế

Bộ thử đầu tiên đề xuất

Tạo khoảng 200–300 câu hỏi được người phụ trách chuyên môn duyệt để phát triển ban đầu, không coi đây là cỡ mẫu đủ để xác nhận an toàn lâm sàng. Mỗi câu có ý định, phạm vi người dùng, evidence span/nguồn đúng, đáp án mong đợi hoặc lý do phải từ chối.

Nhóm câu hỏi	Ví dụ kỹ thuật	Lỗi cần bắt
Tra cứu trực tiếp	Tìm định nghĩa hoặc một mục trong tài liệu.	Bỏ sót thuật ngữ, sai trang nguồn.
Điều kiện và ngoại lệ	Hỏi một khuyến cáo kèm nhóm không áp dụng.	Mất phủ định hoặc đối tượng.
Bảng và đơn vị	Hỏi nội dung một hàng kèm chú thích.	Sai cột, sai đơn vị, mất footnote.
Nhiều nguồn	So sánh hai hướng dẫn hoặc ghép hai mục.	Chỉ tìm được một phần bằng chứng.
Thời gian	Hỏi bản hiện hành và hỏi theo một mốc lịch sử.	Lấy nhầm phiên bản.
Thiếu/xung đột	Câu hỏi ngoài kho hoặc hai nguồn khác nhau.	Bịa đáp án, che giấu mâu thuẫn.
Việt–Anh	Viết tắt, đồng nghĩa, câu hỏi tiếng Việt trên nguồn Anh.	Lỗi dịch hoặc mở rộng truy vấn.
Phân quyền	Người dùng khác quyền hỏi cùng một nội dung.	Rò rỉ qua retrieval, parent, citation hoặc cache.

Tránh đánh giá quá lạc quan

Tách tập theo họ tài liệu, nguồn và phiên bản liên quan; không để bản sao gần giống nằm ở cả nhóm phát triển và test. Với bệnh án, cân nhắc tách theo bệnh nhân/cơ sở/thời gian đúng mục tiêu triển khai. Câu hỏi sinh tự động chỉ là bản nháp; dễ thiên về cách diễn đạt giống nguồn và bỏ qua ca khó.

Chạy ít nhất các baseline: không RAG, lexical-only, dense-only, hybrid và hybrid + reranker. Dùng cùng câu hỏi, snapshot corpus, model sinh và ngân sách context. Báo cáo cả nhóm thất bại, không chỉ điểm trung bình; ghi seed hoặc lặp lại khi kết quả có tính ngẫu nhiên.

Cổng quyết định: thống nhất trước mức recall, citation support, độ trễ và lỗi nghiêm trọng chấp nhận được với người phụ trách sản phẩm/chuyên môn. Không có ngưỡng tự động nào trong báo cáo này đủ để cho phép dùng lâm sàng.

23 · Hệ quả thiết kế và lỗi thường gặp

Sửa đúng tầng gây ra lỗi

Triệu chứng	Kiểm tra trước	Hướng sửa
Không tìm thấy nguồn dù có trong kho	Parser, trạng thái index, bộ lọc và cách viết truy vấn.	Sửa nạp dữ liệu hoặc mở rộng truy vấn có kiểm soát.
Tìm đúng chủ đề nhưng sai nhóm người bệnh	Metadata, nội dung ngoại lệ và reranking.	Giữ đủ điều kiện; thêm kiểm tra applicability.
Top-k gần như giống nhau	Overlap, bản sao và child cùng parent.	Dedup có giữ provenance; điều chỉnh đa dạng nguồn.
Câu trả lời bịa dù retrieval đúng	Context bị cắt, prompt, model và kiểm tra claim.	Giảm nhiễu; ưu tiên evidence; từ chối khi thiếu.
Truy vấn có filter chậm hoặc thiếu kết quả	Query plan và recall dưới filter thực tế.	Tối ưu filter/index, số ứng viên hoặc partition theo nhu cầu.
Bản cũ vẫn xuất hiện	Active generation, cache và sự kiện cập nhật.	Sửa publication workflow và invalidation.
Số liệu tổng hợp sai	Luồng tính toán có đang dùng top-k không?	Chuyển sang SQL/công cụ tính trên dữ liệu đủ phạm vi.

Một suy luận quan trọng cho kiến trúc

Vì dữ liệu nhiều nguồn có chất lượng khác nhau, pipeline cần ghi rõ tài liệu nào đang được phục vụ và vì sao. “Ingest thành công” không nên chỉ có nghĩa file đọc được; phải có trạng thái đã qua kiểm tra và đủ metadata tối thiểu. Những tài liệu chưa duyệt vẫn có thể lưu, nhưng không mặc định đưa vào mọi luồng trả lời.

Quan sát trong vận hành

Theo dõi tỷ lệ no-hit, từ chối, truy vấn có xung đột, citation hỏng, thời gian cập nhật và lỗi theo loại nguồn. Sau khi đổi parser/model/chunker, chạy regression test trước khi publish. Khi người dùng báo lỗi, lưu liên kết tới trace và phiên bản liên quan để tái hiện.

Thêm agent, graph hoặc mô hình lớn hơn có thể hữu ích ở một số lỗi. Nhưng nếu nguyên nhân là mất cột bảng hoặc nhầm quyền truy cập, nâng cấp mô hình sinh thường không xử lý được gốc vấn đề.

24 · Khuyến nghị và lộ trình thực hiện

Bốn giai đoạn, có đầu ra để quyết định

Giai đoạn	Việc chính	Điều kiện chuyển tiếp
1. Chốt phạm vi	Chọn người dùng, nhóm câu hỏi, nguồn được phép; lấy mẫu đại diện.	Có danh mục nguồn và tiêu chí đánh giá được duyệt.
2. Dựng baseline	Parse, version, chunk cấu trúc, Postgres/pgvector, lexical search, citation.	Truy ngược được evidence; chạy được bộ test đầu tiên.
3. Thử nghiệm	So chunk size, parent-child, hybrid, reranker, embedding.	Chọn bằng số đo và phân tích lỗi; không chỉ demo vài câu.
4. Pilot có kiểm soát	Phân quyền, cập nhật/xóa, tải thực, giám sát và phản hồi chuyên gia.	Đạt cổng chất lượng/vận hành cho phạm vi pilot cụ thể.

Mốc thời gian và nhân sự phụ thuộc số nguồn, chất lượng PDF/OCR, quyền dữ liệu và khả năng bố trí người đánh giá. Chưa đủ thông tin để cam kết số tuần triển khai. Trước tiên nên lấy một mẫu có đủ từng loại nguồn thay vì ingest ngay toàn bộ kho.

Cấu hình xuất phát đề xuất

- **Dữ liệu:** object storage giữ file gốc; PostgreSQL giữ document/version/chunk/provenance và trạng thái duyệt.
- **Chunk:** section-aware; thử child 200–400 token, parent 800–1.500 token; bảng có pipeline riêng.
- **Retrieval:** lexical + dense, fusion, reranking; kiểm soát quyền và phiên bản ở mọi bước.
- **Embedding:** thử ít nhất hai ứng viên đa ngôn ngữ, cùng quy trình đánh giá Việt–Anh.
- **Đáp án:** trả lời ngắn, nguồn theo từng ý, nêu thiếu bằng chứng hoặc xung đột.
- **Vận hành:** index generation, cập nhật có kiểm soát, kiểm thử xóa/thu hồi và regression test.

Những câu cần trả lời khi bắt đầu dự án

Ai sẽ dùng và câu trả lời hỗ trợ quyết định gì? Nguồn gồm những định dạng/ngôn ngữ nào? Có bệnh án hay chỉ tài liệu tham khảo? Bao nhiêu tài liệu và cập nhật bao lâu? Yêu cầu on-premise hay cloud? Độ trễ, QPS và ngân sách mong muốn? Ai quyết định nguồn nào được ưu tiên và chấm các ca khó?

Những thông tin này sẽ quyết định điều chỉnh chunk, quyền, DB và cổng đánh giá. Kiến trúc trong báo cáo là cơ sở để bắt đầu đo và ra quyết định có bằng chứng.

25 · Phụ lục A: một chunk mẫu

Ví dụ dưới đây là hợp đồng dữ liệu đề xuất, dùng nội dung giả lập. Trong DB, các trường thường lọc nên được tách thành cột; JSON là cách dễ đọc để trao đổi giữa các bước pipeline.

```
{
  "tenant_id": "demo-team",
  "document_id": "guideline-x",
  "version_id": "guideline-x-v2",
  "chunk_id": "guideline-x-v2-s3-c1",
  "parent_id": "guideline-x-v2-s3",
  "kind": "recommendation",
  "language": "vi",
  "section_path": ["Huong dan X", "Muc 3"],
  "source_text": "Noi dung nguyen ban cua doan...",
  "search_prefix": "Huong dan X; muc 3; nguoi lon",
  "source_location": {
    "pdf_page": 7,
    "printed_page_label": "5",
    "block_ids": ["b18", "b19", "b20"],
    "normalized_text_version": "parse-v1",
    "char_start": 2400,
    "char_end": 2710
  },
  "applicability": {
    "population": "adult",
    "extraction_method": "rule+review",
    "review_status": "approved"
  },
  "source_status": "published",
  "effective_from": "2026-01-01",
  "effective_to": null,
  "content_hash": "sha256:<hash-of-source-span>",
  "embedding_profile_id": "baseline-v1",
  "pipeline_version": "parse-v1/chunk-v1",
  "related_evidence_ids": ["guideline-x-v2-s4"]
}
```

Các chuỗi không dấu trong JSON chỉ nhằm làm ví dụ code gọn; hệ thống thật phải giữ Unicode tiếng Việt. Hash ở đây là placeholder minh họa, không phải dữ liệu thật. Ghi rõ `char_end` là exclusive hay inclusive trong hợp đồng; đề xuất dùng khoảng `[start, end)`.

Hai payload khác nhau: `source_text` dùng để trích dẫn; `embedding input` có thể là `search_prefix + source_text`. Khi cache embedding, hash phải tính trên toàn bộ embedding input, không chỉ `source_text`. Khi lưu/chia sẻ chunk, phải có kiểm soát quyền tương ứng với tài liệu gốc.

26 · Phụ lục B: schema SQL minh họa

Đây là ví dụ rút gọn cho **kho tri thức**, chưa phải migration production. Số chiều 1.024 chỉ để minh họa. Phần chính của báo cáo mô tả thêm sources, parents, ACL và ingestion runs cần bổ sung theo phạm vi thực tế.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
    tenant_id uuid NOT NULL,
    document_id uuid NOT NULL,
    canonical_key text NOT NULL,
    title text NOT NULL,
    PRIMARY KEY (tenant_id, document_id),
    UNIQUE (tenant_id, canonical_key)
);

CREATE TABLE document_versions (
    tenant_id uuid NOT NULL,
    version_id uuid NOT NULL,
    document_id uuid NOT NULL,
    source_uri text NOT NULL,
    content_hash text NOT NULL,
    status text NOT NULL CHECK
        (status IN ('draft','published','withdrawn')),
    effective_from timestamptz,
    effective_to timestamptz,
    PRIMARY KEY (tenant_id, version_id),
    FOREIGN KEY (tenant_id, document_id)
        REFERENCES documents (tenant_id, document_id),
    CHECK (effective_to IS NULL OR
           effective_from IS NULL OR
           effective_to > effective_from)
);

CREATE TABLE chunks (
    tenant_id uuid NOT NULL,
    chunk_id uuid NOT NULL,
    version_id uuid NOT NULL,
    content text NOT NULL,
    locator jsonb NOT NULL,
    content_hash text NOT NULL,
    PRIMARY KEY (tenant_id, chunk_id),
    FOREIGN KEY (tenant_id, version_id)
        REFERENCES document_versions (tenant_id, version_id)
);
```

Khóa ngoại có tenant_id nhằm tránh liên kết nhầm giữa hai tenant. Khi thêm parent_id, ràng buộc cả tenant và version của parent; không để child mở rộng sang parent thuộc phạm vi khác. Với publication thực tế, cần quy tắc không chồng lấn phiên bản hiệu lực cho cùng tài liệu nếu nghiệp vụ yêu cầu một bản hiện hành.

27 · Phụ lục C: vector và quyền truy cập

Ví dụ tiếp nối trang trước. Bảng vector này chỉ chứa **một embedding profile cố định**. Khi đổi model/profile, tạo bảng hoặc index generation khác; profile phải được đăng ký bằng model revision, preprocessing và tokenizer chính xác trước khi dùng.

```
CREATE TABLE chunk_vectors_v1 (
  tenant_id uuid NOT NULL,
  chunk_id uuid NOT NULL,
  embedding vector(1024) NOT NULL,
  PRIMARY KEY (tenant_id, chunk_id),
  FOREIGN KEY (tenant_id, chunk_id)
    REFERENCES chunks (tenant_id, chunk_id)
);

CREATE INDEX chunk_vectors_v1_hnsw
  ON chunk_vectors_v1 USING hnsw
  (embedding vector_cosine_ops);

CREATE INDEX chunks_version_idx
  ON chunks (tenant_id, version_id);

ALTER TABLE chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE chunks FORCE ROW LEVEL SECURITY;
CREATE POLICY chunks_tenant_policy ON chunks
  USING (tenant_id::text =
    current_setting('app.tenant_id', true))
  WITH CHECK (tenant_id::text =
    current_setting('app.tenant_id', true));

ALTER TABLE chunk_vectors_v1 ENABLE ROW LEVEL SECURITY;
ALTER TABLE chunk_vectors_v1 FORCE ROW LEVEL SECURITY;
CREATE POLICY vectors_tenant_policy ON chunk_vectors_v1
  USING (tenant_id::text =
    current_setting('app.tenant_id', true))
  WITH CHECK (tenant_id::text =
    current_setting('app.tenant_id', true));
```

Giới hạn cần hiểu: policy minh họa mới bảo vệ hai bảng và mới theo tenant. Phải áp dụng kiểm soát tương ứng cho documents, versions, parents, file gốc, log và mọi đường đọc khác. Với bệnh án, bổ sung quyền bệnh nhân/encounter; tenant isolation không thay thế các quyền này.

Backend đã xác thực đặt app.tenant_id theo transaction; không lấy nguyên giá trị do người dùng/LLM gửi. Không cho người dùng cuối chạy SQL tùy ý bằng role ứng dụng. Thiết lập GRANT tối thiểu, không dùng superuser/BYPASSRLS và kiểm thử rò rỉ khi pool tái sử dụng kết nối. [\[S15\]](#)

Schema và truy vấn này chưa được chạy trên DB của dự án. Cần kiểm thử migration, kế hoạch truy vấn và phân quyền trước khi triển khai; việc tài liệu được kiểm tra định dạng không phải xác nhận code production.

28 · Phụ lục D: truy xuất có kiểm soát

Pseudocode cho một lượt hỏi

```

auth = authenticate(request)
scope = authorize(auth, requested_task)
snapshot = resolve_published_generation(scope)

intent = classify_question(request.question)
if intent requires exact numbers or aggregation:
    use scoped, parameterized structured-data tools

query = normalize_without_losing_entities_or_negation()
vector = embed(query, snapshot.embedding_profile)

dense = dense_search(vector, scope, snapshot, limit=50)
lexical = lexical_search(query, scope, snapshot, limit=50)
candidates = deduplicate(rrf(dense, lexical, c=60))

# Fetch only authorized, published evidence.
candidates = fetch_and_check(candidates, scope, snapshot)
ranked = rerank(query, candidates[:50])
evidence = expand_context_with_budget(ranked, scope, snapshot)
evidence = recheck_status_and_permissions(evidence)

if evidence is insufficient:
    return clarify_or_abstain_with_reason()

answer = generate_with_source_ids(query, evidence)
answer = validate_citations_and_critical_claims(answer, evidence)
store_minimal_audit_trace(scope, snapshot, evidence.ids)
return answer

```

Chặn tài liệu không được phép ở retrieval hoặc tại lớp cách ly tương đương, rồi xác minh lại khi fetch. Lọc sau khi rerank hoặc sau khi gửi LLM là quá muộn. Hàm insufficient cần quy tắc và ngưỡng đã đánh giá trên bộ test; không chỉ lấy một ngưỡng cosine tùy ý.

Kết quả benchmark cần ghi lại

Mỗi experiment nên có corpus_snapshot, split_version, parser_version, chunk_config, embedding_profile, retrieval_config, reranker, generator, prompt_version và hardware. Kết quả gồm chỉ số theo nhóm câu hỏi, số lần chạy, ngân sách token, latency và chi phí thực.

Để điều tra lỗi, lưu ID bằng chứng ở cả trước và sau rerank, phần context cuối cùng cùng thông tin bị cắt. Nội dung trace nhạy cảm phải có phạm vi quyền và thời hạn giữ riêng. Khi báo cáo kết quả, ghi rõ dữ liệu giả lập hay thật và cấu hình nào chưa được thử.

29 · Phụ lục E: bảng thuật ngữ

Từ	Nghĩa trong báo cáo	Điểm cần nhớ
Grounding	Gắn câu trả lời với bằng chứng truy xuất.	Có evidence chưa đủ; phải diễn đạt đúng evidence.
Hallucination	Sinh thông tin không được hỗ trợ hoặc sai.	RAG có thể giảm, không bảo đảm loại bỏ.
Provenance	Nguồn gốc và lịch sử biến đổi dữ liệu.	Khác với log ai đã truy cập.
Parent-child	Chunk con liên kết tới ngữ cảnh cha.	Đơn vị tìm kiếm và đơn vị đọc có thể khác nhau.
Hybrid retrieval	Kết hợp các đường tìm kiếm bổ sung nhau.	Thường là lexical + dense.
Reranking	Xếp hạng lại tập ứng viên.	Không tìm lại được tài liệu đã bị bỏ sót hoàn toàn.
HNSW / IVFFlat	Các loại index ANN.	Đo tốc độ cùng recall dưới bộ lọc thực tế.
Quantization	Biểu diễn vector bằng ít bit hơn.	Có thể giảm độ chính xác; cần kiểm tra.
RLS / ACL	Chính sách theo hàng / danh sách quyền.	Quyền phải đi theo cả chunk, parent và citation.
Idempotency	Chạy lại cùng thao tác không tạo tác dụng lặp không mong muốn.	Cần cho retry khi nạp dữ liệu.
Tombstone	Dấu đánh dấu đã xóa hoặc thu hồi.	Ngăn index cũ hoặc job retry làm nội dung sống lại.
Ablation	Bỏ/thay một thành phần để đo tác động.	Giúp biết cải thiện đến từ đâu.
Gold evidence	Bằng chứng chuẩn dùng để chấm.	Chấp nhận nhiều nguồn đúng tương đương khi phù hợp.
p95 latency	95% lượt đo có độ trễ không vượt mức này.	Ghi rõ tải, concurrency và cold/warm cache.

Cách đọc mức độ chắc chắn trong tài liệu

“Nghiên cứu/tài liệu mô tả” chỉ thông tin có nguồn liên kết. **“Đề xuất/thử”** là lựa chọn thiết kế được tổng hợp cho tình huống hiện tại. **“Ví dụ/giả định”** là dữ liệu minh họa, chưa được đo trên corpus của dự án.

Báo cáo là nghiên cứu kỹ thuật có định hướng thiết kế, không phải systematic review, hướng dẫn điều trị hay chứng nhận an toàn lâm sàng. Nguồn được chọn tập trung vào nghiên cứu gốc và tài liệu chính thức, kiểm tra ngày 01/10/2026. Chưa có thử nghiệm trên dữ liệu riêng của dự án.

Tài liệu tham khảo

Nguồn sơ cấp và tài liệu chính thức; truy cập ngày 01/10/2026. Bấm tiêu đề để mở nguồn. Các ghi chú giúp hiểu phạm vi áp dụng của từng tài liệu.

[S01] [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)

Lewis và cộng sự · 2020

Bài nghiên cứu nền tảng về kết hợp truy xuất và sinh văn bản; không phải chứng nhận cho sử dụng lâm sàng.

[S02] [Benchmarking Retrieval-Augmented Generation for Medicine](#)

Xiong và cộng sự · 2024, Findings of ACL

MedRAG và MIRAGE: 7.663 câu hỏi thuộc năm bộ dữ liệu QA y khoa. Cần phân biệt với các benchmark khác trùng tên MIRAGE.

[S03] [Comparative Evaluation of Advanced Chunking for Retrieval-Augmented Generation in Large Language Models for Clinical Decision Support](#)

Gomez-Cabello và cộng sự · 2025, Bioengineering 12(11):1194

Thử bốn cách chunk trên 30 câu hỏi sau phẫu thuật nâng mũi với Gemini 1.0 Pro; phạm vi hẹp, không suy rộng thành cấu hình tối ưu cho mọi chuyên khoa.

[S04] [Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models](#)

Günther và cộng sự · 2024; bản v3 năm 2025

Giải thích chunk sau transformer và trước pooling; không đồng nghĩa với embed cả tài liệu thành một vector.

[S05] [Dense X Retrieval: What Retrieval Granularity Should We Use?](#)

Chen và cộng sự · 2023; bản v3 năm 2024

Truy xuất theo proposition; kết quả không tự động bảo đảm giữ được các điều kiện và ngoại lệ y khoa.

[S06] [Lost in the Middle: How Language Models Use Long Contexts](#)

Liu và cộng sự · 2023, được chấp nhận tại TACL

Bằng chứng về ảnh hưởng vị trí thông tin trong context trên các mô hình được thử; không gán cùng mức suy giảm cho mọi mô hình mới.

[S07] [Docling Technical Report](#)

Auer và cộng sự · 2024

Nguồn gốc về xử lý bố cục và cấu trúc bảng PDF. Cần đánh giá riêng trên tài liệu tiếng Việt và scan thực tế.

Tài liệu tham khảo · tiếp

[S08] [Introducing Contextual Retrieval](#)

Anthropic · 2024

Bài kỹ thuật của nhà cung cấp về thêm ngữ cảnh cho chunk và kết hợp tìm kiếm; không phải thử nghiệm y khoa độc lập.

[S09] [M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation](#)

Chen và cộng sự · 2024; bản v5 năm 2025

Nguồn về BGE-M3 và khả năng truy xuất đa ngôn ngữ, dense, sparse, multi-vector.

[S10] [Ragas: Automated Evaluation of Retrieval Augmented Generation](#)

Es và cộng sự · 2023; bản v2 năm 2025

Khung đánh giá nhiều khía cạnh RAG; điểm tự động cần đối chiếu chuyên gia y khoa.

[S11] [RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval](#)

Sarathi và cộng sự · 2024

Cây tóm tắt nhiều mức; dùng để định vị trước khi quay về bằng chứng gốc.

[S12] [From Local to Global: A Graph RAG Approach to Query-Focused Summarization](#)

Edge và cộng sự · 2024

GraphRAG cho tổng hợp trên corpus; không coi quan hệ do LLM trích xuất là quan hệ y khoa đã được xác nhận.

[S13] [pgvector — README chính thức](#)

pgvector contributors · Tài liệu trực tuyến, truy cập 01/10/2026

Exact search, HNSW, IVFFlat, bộ lọc và iterative scans. Chốt lại phiên bản khi triển khai; SQL trong báo cáo là thiết kế minh họa.

[S14] [PostgreSQL 18 — Controlling Text Search](#)

PostgreSQL Global Development Group · Tài liệu PostgreSQL 18

tsvector, tsquery và xếp hạng full-text. ts_rank/ts_rank_cd không phải BM25.

Tài liệu tham khảo · tiếp

[S15] [PostgreSQL 18 — Row Security Policies](#)

PostgreSQL Global Development Group · Tài liệu PostgreSQL 18

RLS và các ngoại lệ như superuser/BYPASSRLS/chủ bảng; cần kiểm thử bằng role ứng dụng thật.

[S16] [Indexing](#)

Qdrant · Tài liệu trực tuyến, truy cập 01/10/2026

Payload index và chỉ mục vector; benchmark phải dùng bộ lọc có độ chọn lọc giống thực tế.

[S17] [Quantization](#)

Qdrant · Tài liệu trực tuyến, truy cập 01/10/2026

Các phương án lượng tử hóa và rescaling; tiết kiệm bộ nhớ phải đo cùng chất lượng truy xuất.

[S18] [Hybrid search](#)

Elastic · Tài liệu trực tuyến, truy cập 01/10/2026

Kết hợp lexical và semantic retrieval; kiểm tra phiên bản và điều kiện cấp phép của tính năng sử dụng.

[S19] [FHIR R4 — Observation](#)

HL7 · FHIR v4.0.1

Mô hình hóa phép đo/quan sát có cấu trúc. Báo cáo dùng R4 làm ví dụ rõ phiên bản, không khẳng định đây là bản mới nhất.

[S20] [FHIR R4 — Provenance](#)

HL7 · FHIR v4.0.1

Truy vết nguồn gốc và tác nhân xử lý dữ liệu; khác với log truy cập hệ thống.

[S21] [PMC Open Access Subset](#)

U.S. National Library of Medicine · Tài liệu trực tuyến, truy cập 01/10/2026

Không phải mọi bài trong PMC đều được phép tái sử dụng; đọc giấy phép từng bài và dùng kênh tải tự động được cho phép.

Tài liệu tham khảo · tiếp

[S22] [Introduction to MeSH](#)

U.S. National Library of Medicine · Tài liệu trực tuyến, truy cập 01/10/2026

Từ vựng kiểm soát phục vụ lập chỉ mục y sinh; dùng như lớp chuẩn hóa bổ sung.

[S23] [LLM Prompt Injection Prevention Cheat Sheet](#)

OWASP · Tài liệu trực tuyến, truy cập 01/10/2026

Chỉ dẫn độc hại trong dữ liệu được truy xuất là một nguồn tấn công; cần phòng vệ nhiều lớp.

[S24] [LLM08:2025 Vector and Embedding Weaknesses](#)

OWASP Gen AI Security Project · 2025

Rủi ro ở vector store, phân quyền, rò rỉ và đầu độc dữ liệu.

[S25] [MRAG: Benchmarking Retrieval-Augmented Generation for Bio-medicine](#)

Liz Li và Wei Zhu · 2026, arXiv v2 — preprint

Benchmark tiếng Anh và Trung; tham khảo hướng mở rộng tác vụ, không phải bằng chứng cho tiếng Việt hay triển khai lâm sàng.

[S26] [Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models](#)

Zhang và cộng sự · 2025

Một họ mô hình embedding/reranking đa ngôn ngữ để đưa vào shortlist thử nghiệm; không mặc định là lựa chọn tốt nhất cho corpus này.